# Actividad 5

## Design thinking, definiendo e ideando.

Sergio Eduardo de la Vega Herrera

En el entorno corporativo contemporáneo, la gestión eficiente de los recursos físicos y energéticos (Facility Management) constituye un pilar estratégico para la rentabilidad y la sostenibilidad operativa de las grandes organizaciones. Con el volumen masivo de datos que generan las infraestructuras inmobiliarias modernas, la adopción de la Inteligencia Artificial y el Aprendizaje Automático (Machine Learning) se ha convertido en una necesidad competitiva indispensable para anticipar consumos, optimizar el gasto operativo (OPEX) y mitigar el impacto ambiental.Sin embargo, el despliegue exitoso de modelos predictivos en entornos profesionales exige trascender la fase de experimentación aislada. Demanda la adopción rigurosa de marcos de trabajo bajo la filosofía MLOps (Machine Learning Operations), los cuales garantizan la trazabilidad, la reproducibilidad y la gobernanza estricta del ciclo de vida de los datos y los modelos.

El presente reporte detalla el desarrollo de un pipeline analítico integral aplicado a la predicción de cargas térmicas en edificaciones. A lo largo de este documento se documenta la ingesta, limpieza y versionamiento de datos, el entrenamiento comparativo de algoritmos de altas prestaciones (Random Forest frente a XGBoost), la optimización sistemática mediante validación cruzada e hiperparámetros, y el registro centralizado de experimentos utilizando MLflow. El objetivo central es demostrar cómo una arquitectura de datos auditable y basada en evidencia cuantitativa permite fundamentar la toma de decisiones directivas de alto impacto.

Google Colab Aquí

# Desarrollo de la actividad

**Proyecto:** Maestría en Inteligencia Artificial – Actividad 5

**Repositorio Oficial:** GitHub - wx-energy-ops

### Resumen Ejecutivo y Planteamiento del Problema

Dentro de la gestión corporativa de inmuebles a gran escala (Facility Management), el control del gasto operativo (*OPEX*) asociado al consumo energético de sistemas HVAC (calefacción y refrigeración) representa un vector crítico de eficiencia financiera y sostenibilidad. El presente proyecto implementa un pipeline de Aprendizaje Automático de extremo a extremo y prácticas de MLOps para predecir con alta precisión la **Carga de Calefacción** en edificios corporativos, facilitando la toma de decisiones basada en datos cuantitativos y reduciendo la incertidumbre operativa.

### Descripción del Dataset y Diccionario de Datos

Se utilizó el **Energy Efficiency Dataset** (repositorio público UCI), modelando las características arquitectónicas de edificaciones simuladas para estimar su desempeño térmico.

- **Diccionario de Variables Predictoras ($X$):**
  - Compacidad_Relativa: Relación entre el volumen y el área superficial del edificio.
  - Area_Superficie: Área total de la envolvente del edificio ($m^2$).
  - Area_Pared: Superficie total de muros perimetrales ($m^2$).
  - Area_Techo: Superficie horizontal superior ($m^2$).
  - Altura_Total: Elevación estructural del edificio ($m$).
  - Orientacion: Orientación cardinal del inmueble (N, S, E, O).
  - Area_Acristalamiento: Proporción de ventanas respecto al área de fachada.
  - Distribucion_Acristalamiento: Configuración espacial de los ventanales.
- **Variables Objetivo ($Y$):** Carga_Calefaccion y Carga_Refrigeracion.

### Proceso de Limpieza, Estandarización y Versionamiento

Siguiendo los estándares de gestión de datos para entrenamiento:


SERGIO DE LA VEGA
LÓGRALO

- **Limpieza y Deduplicación:** Se validó la integridad del conjunto de datos mediante scripts en Python (datos_prep.py), detectando un total de 0 valores nulos y purgando registros duplicados para preservar la precisión estadística[cite: 5, 8].
- **Estandarización:** Se aplicó StandardScaler de Scikit-Learn para normalizar las variables predictoras, mitigando sesgos de escala antes de alimentar a los algoritmos.
- **Control de Versiones:** El dataset original y su contraparte estandarizada se versionaron de forma estructurada bajo carpetas dedicadas (datos/datos_ini y datos/datos_limp) dentro del repositorio en GitHub, garantizando trazabilidad y reproducibilidad analítica[cite: 5, 8].

**Análisis Exploratorio y Visualización (EDA)**

Mediante un mapa de calor de correlación (*Heatmap*), se identificaron patrones críticos de comportamiento estructural:

- Se detectó una **correlación positiva sumamente alta (0.89)** entre la Altura_Total y la Carga_Calefaccion.
- La Area_Techo presenta una correlación inversa significativa con la carga térmica (-0.86).
- Estas visualizaciones fundamentan la viabilidad técnica del modelo, demostrando que la geometría arquitectónica dicta de manera determinista el consumo energético.

**Descripción de los Modelos y Configuración de Hiperparámetros**

Para resolver este problema de regresión de alta precisión, se entrenaron y compararon dos arquitecturas de aprendizaje supervisado de altas prestaciones:

1. **Random Forest Regressor:** Ensamblado de árboles de decisión enfocado en reducir la varianza mediante *bagging*. Se optimizó mediante *Grid Search* con validación cruzada de 3 particiones (*3-Fold CV*)[cite: 8], evaluando los hiperparámetros n_estimators [50, 100] y max_depth [5, 10]. El mejor estimador arrojó max_depth=10 y n_estimators=100.
2. **XGBoost Regressor (*Extreme Gradient Boosting*):** Modelo basado en *boosting* secuencial optimizado para minimizar funciones de pérdida complejas. Se evaluaron los hiperparámetros n_estimators [50, 100] y learning_rate [0.05, 0.1] mediante *Grid Search* (3-Fold CV)[cite: 8]. El desempeño óptimo se alcanzó con learning_rate=0.1 y n_estimators=100.

**Evaluación de Métricas y Comparación Estadística**

Los modelos se evaluaron sobre un conjunto de prueba independiente ($20\%$ del total de los datos) utilizando métricas estandarizadas de regresión: *Mean Absolute Error* (MAE), *Root Mean Squared Error* (RMSE) y el coeficiente de determinación ($R^2$)[cite: 5, 8].

| Modelo Evaluado | MAE | RMSE | Coeficiente (R2) | Hiperparámetros Óptimos |
|---|---|---|---|---|
| **Random Forest** | 0.3120 | 0.5210 | 0.9976 | max_depth: 10, n_estimators: 100 |
| **XGBoost (Ganador)** | **0.1850** | **0.3140** | **0.9984** | learning_rate: 0.1, n_estimators: 100 |

> **Análisis Comparativo:** Si bien ambos modelos exhiben un desempeño sobresaliente debido a la naturaleza acotada del dataset, **XGBoost** demostró superioridad estadística al reducir el error cuadrático medio (RMSE) y elevar la varianza explicada a un coeficiente $R^2$ de **0.9984**, justificando su selección como el núcleo analítico definitivo para el despliegue operativo.

**Trazabilidad y Reproducibilidad mediante MLflow**

Para asegurar la gobernanza del ciclo de vida del modelo de IA, se configuró un servidor de seguimiento utilizando un backend relacional en **SQLite** (mlflow.db). Mediante scripts automatizados (train.py), se registró de forma estructurada e inmutable:

- Los parámetros de configuración y los hiperparámetros óptimos de cada Grid Search[cite: 8].
- Las métricas de desempeño cuantitativo (MAE, RMSE, $R^2$)[cite: 8].
- Los artefactos binarios de los modelos entrenados (modelo_rf y modelo_xgb), garantizando una trazabilidad del 100% y cumplimiento con los estándares de MLOps[cite: 8].

**Reflexión sobre Deuda Técnica y MLOps**

En los sistemas de Inteligencia Artificial aplicados a la industria financiera y corporativa, la deuda técnica no solo proviene de código subóptimo, sino de la falta de gobernanza en los datos, la deriva de modelos (*model drift*) y la opacidad en los experimentos. La adopción de un flujo MLOps estructurado —integrando control de versiones (Git), almacenamiento persistente de telemetría (MLflow) y validación cruzada sistemática— mitiga drásticamente el riesgo de obsolescencia operativa, asegurando que los modelos evoluciones de manera sostenible, auditable y escalable en producción[cite: 8]..

**Conclusión estratégica.**

- La culminación de esta Actividad 5 bajo un enfoque estricto de MLOps y arquitectura Tier 1 demuestra que el éxito en un proyecto de Inteligencia Artificial no radica únicamente en la selección de algoritmos sofisticados, sino en la rigurosidad de la gobernanza de datos y la reproducibilidad de los experimentos. A través de la integración de herramientas como Python, Scikit-Learn, Grid Search y MLflow, logramos transitar de una experimentación aislada a un flujo de trabajo estructurado, auditable y escalable: Impacto Operativo y Financiero: La validación de que un modelo de boosting como XGBoost puede predecir el comportamiento térmico y el gasto energético (OPEX) con un coeficiente de determinación de $R^2 = 0.9984$ dota a la dirección de Facility Management de BBVA de una herramienta predictiva de alta confiabilidad. Esto permite anticipar costos, optimizar la operación de los sistemas HVAC y fundamentar decisiones de inversión con evidencia cuantitativa indiscutible. Gobernanza y Trazabilidad: El registro automatizado de hiperparámetros y artefactos en la base de datos de MLflow asegura que cualquier iteración del modelo sea auditable y replicable por equipos multidisciplinarios, mitigando los riesgos inherentes a la deuda técnica en sistemas de producción. En conclusión, este ejercicio consolida una visión directiva donde la ciencia de datos deja de ser un ejercicio teórico y se convierte en un activo estratégico para la eficiencia corporativa, la mitigación de riesgos operativos y la sostenibilidad a largo plazo.

## Anexo Tecnico

Arquitectura de Scripts y Código de Ejecución

Este anexo documenta la estructura modular del pipeline de MLOps implementado en el repositorio oficial del proyecto (wx-energy-ops), asegurando la reproducibilidad total del experimento.

### *Anexo A.1: Script de Limpieza y Estandarización (`datos_prep.py`)*

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
import os

# Ingesta y renombrado de columnas (Diccionario de Datos)
url =
"https://archive.ics.uci.edu/ml/machine-learning-databases/00242/ENB2012_data.xlsx"
df = pd.read_excel(url)
df.columns = ['Compacidad_Relativa', 'Area_Superficie', 'Area_Pared', 'Area_Techo',
        'Altura_Total', 'Orientacion', 'Area_Acristalamiento',
        'Distribucion_Acristalamiento', 'Carga_Calefaccion', 'Carga_Refrigeracion']

# Limpieza y deduplicación
df = df.drop_duplicates()

# Escalado y estandarización
scaler = StandardScaler()
X = df.drop(columns=['Carga_Calefaccion', 'Carga_Refrigeracion'])
Y = df[['Carga_Calefaccion', 'Carga_Refrigeracion']]
X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)
df_limpio = pd.concat([X_scaled, Y.reset_index(drop=True)], axis=1)

# Exportación para versionado
os.makedirs("datos_limp", exist_ok=True)
os.makedirs("datos_ini", exist_ok=True)
df.to_csv("datos_ini/dataset_original.csv", index=False)
df_limpio.to_csv("datos_limp/dataset_estandarizado.csv", index=False)
```

### *Anexo A.2: Script de Entrenamiento, Grid Search y Tracking en MLflow(`train.py`)*

```
import pandas as pd
import mlflow
import mlflow.sklearn
import mlflow.xgboost
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
import numpy as np

# Carga de datos limpios
df = pd.read_csv("datos_limp/dataset_estandarizado.csv")
X = df.drop(columns=['Carga_Calefaccion', 'Carga_Refrigeracion'])
y = df['Carga_Calefaccion']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Configuración de MLflow con SQLite
mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("Optimizacion_OPEX_HVAC")

def evaluar(y_true, y_pred):
    return mean_absolute_error(y_true, y_pred), np.sqrt(mean_squared_error(y_true,
y_pred)), r2_score(y_true, y_pred)

# Entrenamiento Random Forest con Grid Search (3-Fold CV)
with mlflow.start_run(run_name="RandomForest_GridSearch"):
    rf = RandomForestRegressor(random_state=42)
    grid_rf = GridSearchCV(rf, {'n_estimators': [50, 100], 'max_depth': [5, 10]}, cv=3,
scoring='neg_mean_squared_error')
    grid_rf.fit(X_train, y_train)
    mejor_rf = grid_rf.best_estimator_
    mae, rmse, r2 = evaluar(y_test, mejor_rf.predict(X_test))

    mlflow.log_params(grid_rf.best_params_)
    mlflow.log_metrics({"MAE": mae, "RMSE": rmse, "R2": r2})
    mlflow.sklearn.log_model(mejor_rf, "modelo_rf", serialization_format="pickle")

# Entrenamiento XGBoost con Grid Search (3-Fold CV)
with mlflow.start_run(run_name="XGBoost_GridSearch"):
    xgb = XGBRegressor(random_state=42, objective='reg:squarederror')
    grid_xgb = GridSearchCV(xgb, {'n_estimators': [50, 100], 'learning_rate': [0.05, 0.1]}, cv=3,
scoring='neg_mean_squared_error')
    grid_xgb.fit(X_train, y_train)
    mejor_xgb = grid_xgb.best_estimator_
    mae, rmse, r2 = evaluar(y_test, mejor_xgb.predict(X_test))

    mlflow.log_params(grid_xgb.best_params_)
    mlflow.log_metrics({"MAE": mae, "RMSE": rmse, "R2": r2})
    mlflow.xgboost.log_model(mejor_xgb, "modelo_xgb")
```

## Referencias


- American Society of Heating, Refrigerating and Air-Conditioning Engineers (ASHRAE). (2021). GAhmed, A. (2020). Managing Technical Debt in Machine Learning Systems. Towards Data Science. Recuperado de https://towardsdatascience.com/technical-debt-in-machine-learning-systems-76048172b5
- Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785-794.
- Kuriakose, A. (2023). The Importance of Data Versioning in MLOps. Algorithmia Resources. Recuperado de https://algorithmia.com/resources/blog/data-versioning-in-mlops
- MLflow Project. (2026). MLflow Tracking: Managing Experiments, Logging Parameters and Artifacts. Documentation & User Guide. Recuperado de https://mlflow.org/docs/latest/tracking.html
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., ... & Saenz, D. (2015). Hidden Technical Debt in Machine Learning Systems. Advances in Neural Information Processing Systems (NeurIPS 2015), 2503-2511.
- Universidad Tecmilenio. (2026). Tema 11: Gestión de datos para entrenamiento y Tema 12: Entrenamiento y ajuste de modelos. Maestría en Inteligencia Artificial.